Design Patterns

Selim Uysal

Ankara, Turkey

Abstract

In this article, we discuss a topic that is crucial for programmers: how to write better code. The article consists of four sections: introduction, design patterns, anti-patterns and conclusion. First, we explain the SOLID principles and then examine design patterns, including their advantages and disadvantages. We explore each design pattern in detail and give an example that illustrates its application. Finally, we explain what anti-patterns are.

1. Main text

When developing software, it is recommended to apply the SOLID principles to improve the readability, maintainability and flexibility of the code.

- Single Responsibility Principle: A class or function should have only one responsibility.
- Open/Closed Principle: Classes and functions should be open for extension but closed for modification.
- Liskov Substitution Principle: Derived classes should be substitutable for their base classes.
- Interface Segregation Principle: Classes should not be forced to depend on interfaces they do not use.
- Dependency Inversion Principle: High-level modules should not depend on low-level modules; both should depend on abstractions.

These five principles should be applied to all code. In addition to them, there are design patterns, which are well-known, proven code structures.

2. Design patterns: Design patterns are reusable solutions to common challenges in object-oriented software development. These solutions have been developed over time to address recurring problems. Design patterns are not libraries; they are code structures that are written within your own code. They can be categorized into three types: creational, structural and behavioral patterns.

Advantages:

- Reduces design errors and repetitive code.
- Increases the reusability of the code.
- Makes the design flexible, extensible and sustainable.

Disadvantages:

- In some cases, it may cause unnecessary complexity.
- It may increase the cost, since additional classes and code are required.

2.1. Creational Patterns: These patterns deal with object creation and the process of creating objects.

Factory Method Pattern: Object creation is handled in a single place (a factory method), and the other classes use the objects it produces. Its most important feature is that it makes the code flexible by letting subclasses decide which objects are created. It is usually used when the exact types of the objects to be created are not known in advance. The most important advantage is that when a new type is added, the code continues to work by adding only the necessary class instead of changing the whole code.

```cpp
#include <iostream>
using namespace std;

class Araba {
public:
    string getTip() {
        return "Bilinmeyen Araba Tipi";
    }
};

class Hatchback : public Araba {
public:
    string getTip() {
        return "Hatchback Tipinde Araba Seçildi.";
    }
};

class Sedan : public Araba {
public:
    string getTip() {
        return "Sedan Tipinde Araba Seçildi.";
    }
};

class ArabaFactory {
public:
    static Araba* getAraba(string tip) {
        if (tip == "Sedan")
            return new Sedan();
        else if (tip == "Hatchback")
            return new Hatchback();                    ;

        return nullptr;
    }
};
    delete hatchbackAraba;
    delete sedanAraba;

    return 0;
}
```

EXAMPLE: There is a car class (Araba); the Hatchback and Sedan classes inherit from it and override its function. The objects are produced from a single class: the getAraba function of the ArabaFactory class creates a hatchback or a sedan according to the type string, using if/else conditions. In main, the objects are created as pointers by the factory class, allocating memory dynamically. If the team that wrote this code wants to produce a new car model, they can add it by adding one new class (and one branch in the factory) without changing the rest of the code, which gives the code flexibility.

Abstract Factory Pattern: Because it looks similar to the Factory Method, we explain the difference through an example. The Factory Method leaves the object creation function to subclasses, whereas the

Abstract Factory allows families of related objects to be created through an abstract class. In this way, users interact only with abstract classes and do not depend on concrete classes. The most important advantage is flexibility, because the dependencies in the code are reduced.

[*] Untitled2

```cpp
#include <iostream>
using namespace std;

class Araba {
public:
    virtual string getTip() = 0;
};

class Hatchback : public Araba {
public:
    string getTip() {
        return "Hatchback Tipinde Araba Seçildi.";
    }
};

class Sedan : public Araba {
public:
    string getTip() {
        return "Sedan Tipinde Araba Seçildi.";
    }
};

class ArabaFactory {
public:
    virtual Araba* getAraba() = 0;
};

class HatchbackFactory : public ArabaFactory {
public:
    Araba* getAraba() {
        return new Hatchback();
    }
};

class SedanFactory : public ArabaFactory {
public:
    Araba* getAraba() {
        return new Sedan();
    }
};
```

```cpp

int main() {
    ArabaFactory* factory = new HatchbackFactory();
    Araba* hatchbackAraba = factory->getAraba();
    cout << hatchbackAraba->getTip() << endl;

    delete factory;
    delete hatchbackAraba;

    factory = new SedanFactory();
    Araba* sedanAraba = factory->getAraba();
    cout << sedanAraba->getTip() << endl;

    delete factory;
    delete sedanAraba;

    return 0;
}
```

EXAMPLE: A customized version of the previous example is used for the Abstract Factory: an abstract car class is used, with a different factory class for each type. The Abstract Factory is usually used when we create products that consist of several related objects, while the Factory Method is applied when we create one kind of object in several variants.

Builder Pattern: When objects of the same class must be created with small differences, constructors can cause problems. The Builder lets us specify the properties of an object separately and create it step by step, which is its main advantage. On the other hand, the complexity of the code increases because several additional classes are needed.

```cpp
#include <iostream>
#include <string>

using namespace std;

class Tost {
public:
    void setEkmek(const string& ekmek) {
        ekmek_ = ekmek;
    }

    void setSos(const string& sos) {
        sos_ = sos;
    }

    void setIcMalzeme(const string& icMalzeme) {
        icMalzeme_ = icMalzeme;
    }

    void gosterTost() {
        cout << "Tost: " << ekmek_ << " ekmek, " << sos_ << " sos, " << icMalzeme_ << " iç malzeme\n";
    }

private:
    string ekmek_;
    string sos_;
    string icMalzeme_;
};
```

```cpp
class AciliTostBuilder : public TostBuilder {
public:
    void ekmekKoy() {
        tost_->setEkmek("kepekli");
    }

    void sosKoy() {
        tost_->setSos("aci sos");
    }

    void icMalzemeKoy() {
        tost_->setIcMalzeme("kasar, acili sucuk, yesil biber");
    }
};

class TostUstasi {
public:
    void setTostBuilder(TostBuilder* builder) {
        tostBuilder_ = builder;
    }

    Tost* getTost() {
        return tostBuilder_->getTost();
    }

    void tostHazirla() {
        tostBuilder_->yeniTostOlustur();
        tostBuilder_->ekmekKoy();
        tostBuilder_->sosKoy();
        tostBuilder_->icMalzemeKoy();
    }

private:
    TostBuilder* tostBuilder_;
};
```

EXAMPLE: A good everyday example is

```cpp
int main() {
    TostUstasi tostUstasi;
    KlasikTostBuilder klasikTostBuilder;
    AciliTostBuilder aciliTostBuilder;

    tostUstasi.setTostBuilder(&klasikTostBuilder);
    tostUstasi.tostHazirla();
    Tost* klasikTost = tostUstasi.getTost
```

```cpp
    virtual void sosKoy() = 0;
    virtual void icMalzemeKoy() = 0;

protected:
    Tost* tost_;
};

class KlasikTostBuilder : public TostBuilder {
public:
    void ekmekKoy() {
        tost_->setEkmek("beyaz");
    }

    void sosKoy() {
        tost_->setSos("mayonez");
    }

    void icMalzemeKoy() {
        tost_->setIcMalzeme("ksar, sucuk, sosis");
    }
};
```

a waiter taking orders in a restaurant: everyone wants their toast prepared differently, for example with different fillings such as sausage. We solve this problem with a builder, and the characteristics of the toast to be created are separated into the toast builder (TostBuilder) classes.

Singleton Pattern: It allows only one object of a class to be created and the same object to be used again and again in the rest of the code. The advantage is that no memory is wasted on unnecessary objects. The disadvantage is that it reduces the testability of the code.

```cpp
#include <iostream>
using namespace std;

class Singleton {
    private:
    static Singleton* instance;
    int data;

    Singleton() {
        data = 0;
    }

public:
    static Singleton* getInstance() {
        if (instance == NULL) {
```

EXAMPLE: We block external access to the Singleton class by making its constructor private, and the object is created only through a function inside the class. Inside this function, the object is created only if it does not exist yet (that is, if the pointer is NULL). The instance pointer is a static member, so it must be defined outside the class, and the function that returns it must also be static.

```cpp
#include <iostream>
#include <string>

using namespace std;

class Asker {
private:
    string isim;

public:
    Asker(string isim) : isim(isim) {}

    string getIsim() {
        return isim;
    }

    Asker* kopyaOlustur() {
        return new Asker(*this);
    }
};

int main() {
    Asker* orijinalAsker = new Asker("Ahmet");
    cout << "Orijinal Asker: " << orijinalAsker->getIsim() << endl;

    Asker* kopyaAsker1 = orijinalAsker->kopyaOlustur();
    cout << "Kopya Asker 1: " << kopyaAsker1->getIsim() << endl;

    Asker* kopyaAsker2 = orijinalAsker->kopyaOlustur();
    cout << "Kopya Asker 2: " << kopyaAsker2->getIsim() << endl;

    delete orijinalAsker;
    delete kopyaAsker1;
    delete kopyaAsker2;

    return 0;
}
```

Prototype Pattern: Instead of creating new objects from scratch, it asks us to copy existing ones. Copying objects in the usual way may cause errors with some private members, so a copy constructor is used. The advantage is that complex objects can be produced

quickly by copying, without depending on their classes. On the other hand, copying objects can sometimes be difficult.

EXAMPLE: To keep the example as simple and understandable as possible, we use the soldiers of a game. After using the new operator only twice in the code, we can create 100 soldiers in the same way by copying them.

Object Pool Pattern: As the name suggests, it works like a pool: the desired number of objects is created and kept in the pool, an object is handed out when it is requested and is returned to the pool when it is released. The advantage is that it saves us from excessive memory use and repeated object creation. The size of the pool must be

```
37  };
38
39  int main() {
40      ObjectPool pool(10);
41
42      Mermi* mermi1 = pool.al();
43      mermi1->atesle();
44
45      Mermi* mermi2 = pool.al();
46      mermi2->atesle();
47
48      pool.geriVer(mermi1);
49      pool.geriVer(mermi2);
50
51      return 0;
52  }
53
```

set well; otherwise there is no point in using an object pool, because producing objects again takes time when the pool is empty.

```cpp
#include <iostream>
#include <vector>

using namespace std;
class Mermi {
public:
    void atesle() {
        cout << "Mermi ateslendi" << endl;
    }
};

class ObjectPool {
private:
    vector<Mermi*> havuz;
    int mks;

public:
    ObjectPool(int boyut) : mks(boyut) {
        for (int i = 0; i < mks; i++) {
            havuz.push_back(new Mermi());
        }
    }

    Mermi* al() {
        if (havuz.empty()) {
            return new Mermi();
        } else {
            Mermi* mermi = havuz.back();
            havuz.pop_back();
            return mermi;
        }
    }

    void geriVer(Mermi* mermi) {
        havuz.push_back(mermi);
    }
};
```

EXAMPLE: A bullet in a game is an object, and as many bullets as desired are created in advance and stored in a pool (a vector). A bullet is taken from the pool with the al() function, using the add and remove functions of the vector class. When the bullet is no longer needed, it is returned to the pool with the geriVer() function.

2.2. Structural Patterns: These patterns deal with the relationships and structures between classes and objects.

Adapter Pattern: As the name suggests, it allows two classes that are not compatible with each other to work together. It resolves problems that could otherwise break the application by adding an adapter between our code and a class that has a different interface. Advantage: it ensures that classes that are different and incompatible, but need to work together, operate smoothly. Disadvantage: converting between dissimilar classes may increase the processing time.

```cpp
#include <iostream>

class Kodunsinifi {
public:
    virtual void adaptr() = 0;
};

class Disardakisinif {
public:
    void eklenen() {
      cout << "Mevcut operasyon calisti." << endl;
    }
};

class Adaptor : public kodunsinifi {
private:
    Disardakisinif d_sinif;

public:
    void adaptr() {
        d_sinif.eklenen();
    }
};

int main() {
    Kodunsinifi* adaptor = new Adaptor();
    adaptor->adaptr();

    delete adaptor;

    return 0;
}
```

EXAMPLE: There are two classes, one from our own code and one external class, and both must be used in the code. We used the Adapter pattern so that they do not conflict and cause an error: in main, we run the adapter object created from the adapter class, which calls the function of the external class through its own function.

Bridge Pattern: Unlike the Adapter, it decouples an abstraction from its

```cpp
class Sekil {
protected:
    UygulamaArayuzu* uygulamaArayuzu;

public:
    Sekil(UygulamaArayuzu* arayuz) : uygulamaArayuzu(arayuz) {}

    virtual void ciz() = 0;
};

class KareSekli : public Sekil {
public:
    KareSekli(UygulamaArayuzu* arayuz) : Sekil(arayuz) {}

    void ciz() {
        uygulamaArayuzu->ciz();
    }
};

class DaireSekli : public Sekil {
public:
    DaireSekli(UygulamaArayuzu* arayuz) : Sekil(arayuz) {}

    void ciz() {
        uygulamaArayuzu->ciz();
    }
};
```

implementation. The Bridge pattern reduces the dependency between the abstraction and the implementation classes that comes from an "is-a" (inheritance) relationship, by connecting them through a "has-a" (composition) relationship, which provides flexibility. Its advantage is that the abstraction and the implementation can be separated and become independent of each other. However, since two separate abstractions are created, the code complexity increases.

```cpp
#include <iostream>
using namespace std;



class UygulamaArayuzu {
public:
    virtual void ciz() = 0;
};

class Kare : public UygulamaArayuzu {
public:
    void ciz() {
        cout << "Kare çizildi." << endl;
    }
};

class Daire : public UygulamaArayuzu {
public:
    void ciz() {
        cout << "Daire çizildi." << endl;
    }
};

int main() {
    UygulamaArayuzu* kareArayuzu = new Kare();
    Sekil* kareSekli = new KareSekli(kareArayuzu);
    kareSekli->ciz();

    UygulamaArayuzu* daireArayuzu = new Daire();
    Sekil* daireSekli = new DaireSekli(daireArayuzu);
    daireSekli->ciz();


    return 0;
}

```

EXAMPLE: The shape (Sekil) is an abstract class from which the square and circle classes inherit. In the same way, the implementation (application interface) is an abstract class with its own square and circle classes. The Bridge pattern connects the shape and the implementation, so a new shape or a new implementation can be added easily and independently.

Composite Pattern: It allows objects to be combined into a hierarchy (a tree structure). The advantage of this pattern is that, because they implement the same interface, the client can manage derived classes and container objects in a single way. In addition, using leaf objects and container objects together makes it easier to create complex structures and provides structural flexibility. The individual objects are usually called "leaves".

```cpp
using namespace std;
class Sekil {
public:
    virtual void ciz() = 0;
};

class Kare : public Sekil {
public:
    void ciz() {
        cout << "Kare çizildi." << endl;
    }
};

class Daire : public Sekil {
public:
    void ciz() {
    cout < < "Daire çizildi." << endl;
    }
};
```

```cpp
class SekilGrubu : public Sekil {
private:
    string ad;
vector<Sekil*> sekiller;

public:
    SekilGrubu( string& isim) : ad(isim) {}

    void ciz() {
        cout  << "Sekil Grubu: " << ad << stndl;

        for (Sekil* sekil : sekiller) {
            sekil->ciz();
        }
    }

    void ekle(Sekil* sekil) {
        sekiller.push_back(sekil);
    }

    void sil(Sekil* sekil) {
    }
};

int main() {
    Sekil* kare = new Kare();
    Sekil* daire = new Daire();

    SekilGrubu* grup = new SekilGrubu("Ana Grup");
    grup->ekle(kare);
    grup->ekle(daire);

    grup->ciz();

    delete kare;
    delete daire;
    delete grup;

    return 0;
```

EXAMPLE: A shape group (SekilGrubu) is a composite structure that combines classes with different names that perform the same function into a single class and presents them as a whole. A recursive function with a for loop allows access to all the leaves of the structure.

Decorator Pattern: Without changing the structure of the class, it can add and remove features of an existing object while the program is running, and it stores the existing object within itself. Each decorator class has a single responsibility, which makes the code clearer and easier to understand, and it allows new features to be added to an object dynamically at runtime. The disadvantage is that a decorator class must

be created for each new feature, and because behavior can be changed dynamically, the code can become harder to understand and follow.

```cpp
#include <iostream>
#include <string>

using namespace std;

class Araba {
public:
    virtual string getModel() const = 0;
    virtual double getPrice() const = 0;
};

class StandartAraba : public Araba {
public:
    string getModel() {
        return "Standart Araba";
    }

    double getPrice() {
        return 50000.0;
    }
};

class ArabaOzellikEkleyici : public Araba {
protected:
    Araba* araba;

public:
    ArabaOzellikEkleyici(Araba* araba) : araba(araba) {}

    string getModel()  {
        return araba->getModel();
    }

    double getPrice() {
        return araba->getPrice();
    }
};
```

```cpp
class HizliAraba : public ArabaOzellikEkleyici {
public:
    HizliAraba(Araba* araba) : ArabaOzellikEkleyici(araba) {}

    string getModel() {
        return araba->getModel() + ", Hizli";
    }

    double getPrice() {
        return araba->getPrice() + 10000.0;
    }
};

class LuksAraba : public ArabaOzellikEkleyici {
public:
    LuksAraba(Araba* araba) : ArabaOzellikEkleyici(araba) {}

    string getModel() {
        return araba->getModel() + ", Lüks";
    }

    double getPrice(){
        return araba->getPrice() + 20000.0;
    }
};
```

EXAMPLE: There is an abstract car class and a standard car class with normal features. After a car is first created as standard, it can be made luxury or fast: if the customer wants to add a feature, the car can be customized through the decorator classes of the car customizer.

Facade Pattern: It allows us to use several complex classes through a single, simple structure.
Advantage: it reduces the amount of code and repetition for the client, and thus reduces the cost.
Disadvantage: changes in the underlying classes can affect the Facade, and it can limit the flexibility of the code.

```cpp
#include <iostream>
using namespace std;

class GuvenlikSistemi {
public:
    void kontrolEt() {
        cout << "Güvenlik sistemi kontrol ediliyor." << endl;
    }
};

class OdemeSistemi {
public:
    void odemeYap() {
        cout << "Ödeme sistemi ile ödeme yapýlýyor." << endl;
    }
};

class EnvanterSistemi {
public:
    void stokKontrolu() {
        cout << "Envanter yönetim sistemi stoklarý kontrol ediyor." << endl;
    }
};
```

EXAMPLE: We turned a shopping center into a Facade by combining complex operations (security, payment, inventory) under one class, which makes things convenient for the client.

Flyweight Pattern: When similar objects are needed, it does not create new objects; it lets existing objects be shared and reused. There are two important parts: the intrinsic state is the part shared by the objects, and the extrinsic state is the part that varies and is stored separately. Advantage: it saves memory. Disadvantage: sharing objects can increase the complexity of the application. Because it is similar to the Object Pool, the most important difference should be kept in mind: the Object Pool only stores and hands out objects, whereas the Flyweight manages the shared and the variable properties of objects. Another difference is that in an object pool the desired number of objects is created at the beginning and then used, whereas in the Flyweight individual objects are not created for each user; all users share a common object.

```cpp
using namespace std;

class Mermi {
public:
    virtual void atesle() = 0;
};

class GercekMermi : public Mermi {
public:
    void atesle() {
        cout << "Gerçek mermi ateslendi" << endl;
    }
};

class HavaMermisi : public Mermi {
public:
    void atesle() {
        cout << "Hava mermi ateslendi" << endl;
    }
};

class MermiFabrikasi {
private:
    vector<Mermi*> havuz;

public:
    Mermi* al(string tür) {
        for (Mermi* mermi : havuz) {
            if (mermi->getType() == tür) {
                havuz.erase(mermi);
                return mermi;
            }
        }

        Mermi* mermi;
        if (tür == "gerçek") {
            mermi = new GercekMermi();
        } else if (tür == "hava") {
            mermi = new HavaMermisi();
        } else {
            cout << "Geçersiz mermi türü" << endl;
            return NULL;
        }

        return mermi;
    }

    void geriVer(Mermi* mermi) {
        havuz.push_back(mermi);
    }
};
```

EXAMPLE: The bullet factory in the Flyweight example should not be confused with the pool we wrote earlier. Although it is similar to the Object Pool, the working logic is different: if an object of the requested type already exists, it is taken and returned; if it does not exist, it is created and stored in the requested type while the program is running.

```cpp
int main() {
    MermiFabrikasi mermiFabrikasi;

    Mermi* mermi1 = mermiFabrikasi.al("gerçek");
    mermi1->atesle();

    Mermi* mermi2 = mermiFabrikasi.al("hava");
    mermi2->atesle();

    mermiFabrikasi.geriVer(mermi1);
    mermiFabrikasi.geriVer(mermi2);

    return 0;
}
```

Proxy Pattern: It acts as an intermediary for the real object and allows us to see or use only the features we want, instead of accessing an object that has many features. The Decorator and the Proxy can be confused: the Decorator is used to add new behavior to an object dynamically, whereas the Proxy controls access to the real object. Its advantage is that it controls access to the object, so resource usage can be optimized. Disadvantage: it may affect performance, because extra code is used to access the object.

```
5  public:
6      virtual void sur() = 0;
7  };
8
9  class GercekAraba : public Araba {
10 private:
11     string model;
12
13 public:
14     GercekAraba(string model) : model(model) {}
15
16     void sur() {
17         cout << model << " arabas sürülüyor." << endl;
18     }
19 };
20
21 class ProxyAraba : public Araba {
22 private:
23     GercekAraba* gercekAraba;
24     string model;
25
26 public:
27     ProxyAraba(string model) : model(model), gercekAraba(NULL) {}
28
29     void sur() {
30         if (gercekAraba == NULL) {
31             gercekAraba = new GercekAraba(model);
32         }
33         cout << "Proxy " << model << " araba denetleniyor." << endl;
34         gercekAraba->sur();
35     }
36 };
37
38 int main() {
39     ProxyAraba proxy("BMW");
40     proxy.sur();
41
42     return 0;
43 }
```

EXAMPLE: When we want to drive a car, we may first want to check whether the car exists. We use a proxy car for this: after the request is sent to the proxy car, it creates the real car and performs the driving operation only when necessary. In this way, we increase performance by preventing unnecessary creation and operations, and we use resources effectively. To avoid confusion: the proxy only controls access to the real car, and it is the real car that is actually driven.

2.3. Behavioral Patterns: These patterns deal with the communication between classes and objects and allow this communication to be organized in more flexible structures.

Chain of Responsibility Pattern: It decouples the classes from each other. In short, a request is passed to the connected classes one after another: each class performs its operation and then passes the request on to the next linked class. In other words, it works like a control mechanism, and the classes are loosely coupled. It is usually used where there would otherwise be many if-else blocks. Advantages: each link of the chain is independent and can be changed; it is easy to add or remove a processing step; and since each class performs a single function, the code is easy to maintain and test. Disadvantages: each processor in the chain is checked in order to handle the request, which can affect performance when the request has to travel to the end of the chain; and if the operation did not take place in any of the classes, it may be difficult to find out why.

```cpp
class BankaIslemi {
protected:
    BankaIslemi* sonrakiIslem;

public:
    BankaIslemi() : sonrakiIslem(NULL) {}

      void setSonrakiIslem(BankaIslemi* islem) {
        sonrakiIslem = islem;
    }

   virtual void talebiIsle(string talep) = 0;
};

class ParaCekme : public BankaIslemi {
public:
    void talebiIsle(string talep) {
        if (talep == "paraCekme ") {
            cout << "Para çekme islemi gerçeklestiriliyor." <<
        }
         else if (sonrakiIslem != NULL)  {
            sonrakiIslem->talebiIsle(talep);
        }
    }
};

 class ParaYatirma : public BankaIslemi {
public:
    void talebiIsle(string talep) {
        if (talep == "paraYatirma") {
            cout << "Para yatirma islemi gerçeklestiriliyor."
        }
         else if (sonrakiIslem != NULL) {
            sonrakiIslem->talebiIsle(talep);
        }
    }
};
```

EXAMPLE: The bank transaction classes (for example withdrawal, deposit and balance inquiry) are linked to each other through a request function. The next pointer being NULL works as an end-of-chain flag: it tells the program that the chain of transactions has ended.

```cpp
class BakiyeSorgulama : public BankaIslemi {
public:
    void talebiIsle(string talep) {
        if (talep == "bakiyeSorgulama") {
            cout << "Bakiye sorgulama islemi gerçeklestiriliyor." << endl;
      }
        else if (sonrakiIslem != NULL)   {
            sonrakiIslem->talebiIsle(talep);
        }
    }
};

int main() {
    BankaIslemi* paraCekme = new ParaCekme();
    BankaIslemi* paraYatirma = new ParaYatirma();
    BankaIslemi* bakiyeSorgulama = new BakiyeSorgulama();

    paraCekme->setSonrakiIslem(paraYatirma);
    paraYatirma->setSonrakiIslem(bakiyeSorgulama);

    paraCekme->talebiIsle("paraYatirma");
    paraCekme->talebiIsle("bakiyeSorgulama");
    paraCekme->talebiIsle("paraCekme");
    return 0;
}
```

Command Pattern: It turns each operation into a separate class. Advantages: it separates the classes from each other and makes operations independent, and new operations are easy to add. Disadvantage: a class is created for each operation, which can increase the cost for the program. In the Chain of Responsibility pattern, requests move along the chain and each object checks whether it can process the request; in the Command pattern, the client sends the request directly to the class that handles it.

```cpp
class Islem {
public:
    virtual void gerceklestir() = 0;
};

class YemekSiparisi : public Islem {
private:
    string yemekAdi;

public:
    YemekSiparisi(string yemekAdi) : yemekAdi(yemekAdi) {}

    void gerceklestir() {
        cout << "Yemek  alindi: " << yemekAdi << endl;
    }
};

 class IcecekSiparisi : public Islem {
private:
    string icecekAdi;

public:
    IcecekSiparisi(string icecekAdi) : icecekAdi(icecekAdi) {

    void gerceklestir() {
        cout << "icecek   alindi: " << icecekAdi << endl;
     }
};
```

EXAMPLE: Unlike Chain of Responsibility, every branch of the if-else logic is created as a class (for example food order and drink order). The operations are passed by the client to the required class, and the necessary work is performed inside that class.

```cpp

  class KomutYurutucu {
private:
    vector<Islem*> komutlar;

public:
    void komutEkle(Islem* komut) {
        komutlar.push_back(komut);
    }

    void komutlariCalistir() {
        for (Islem* komut : komutlar) {
            komut->gerceklestir();
        }
    }
};

int main() {

    Islem* yemekSiparisi = new YemekSiparisi("Pizza");
    Islem* icecekSiparisi = new IcecekSiparisi("Kola");

    KomutYurutucu yurutucu;
    yurutucu.komutEkle(yemekSiparisi);
    yurutucu.komutEkle(icecekSiparisi);

    yurutucu.komutlariCalistir();

    return 0;
}

```

Template Method Pattern: A template of an algorithm is created in the base class, and derived classes can change certain steps of this algorithm without changing its structure. Advantages: it saves code repetition, and individual steps can be changed smoothly and easily. Disadvantage: the code is less flexible, because the derived classes are tied to the base class and can only customize certain steps.

EXAMPLE: A pizza template is defined in the base Pizza class, with steps such as preparing the dough, adding ingredients, baking and serving. The ItalianPizza and TurkPizza classes are derived from this template. Since both classes inherit from Pizza, they can customize the functions of the base class, here the step that adds the ingredients.

```
class Pizza {
public:
    void hazirla() {
        hamurHazirla();
        malzemeleriEkle();
        pisir();
        servisEt();
    }

    virtual void hamurHazirla() {
        cout << "Hamur hazilanir "<<endl;
    }

    virtual void malzemeleriEkle() {
        cout << "Malzemeler ekleniyor." << end
    }

    virtual void pisir() {
        cout << "Pisiriliyor." << endl;
    }

    void servisEt() {
        cout << "Pizza servis ediliyor." << en
    }
};
```

```
class ItalyanPizza : public Pizza {
public:
    void malzemeleriEkle() {
        cout << "Italyan usulu malzemeler ekleniyor." << endl;
    }
};

class TurkPizza : public Pizza {
public:
    void malzemeleriEkle() {
        cout << "Turk usulu malzemeler ekleniyor." << endl;
    }
};

int main() {
    Pizza* italyanPizza = new ItalyanPizza();
    italyanPizza->hazirla();

    cout << endl;

    Pizza* turkPizza = new TurkPizza();
    turkPizza->hazirla();

    return 0;
}
```

Iterator Pattern: It is used to navigate through a collection and access the values in it. The Iterator hides the internal structure of the collection and provides an interface for accessing its elements. Advantages: it gives flexibility to the code by abstracting the internal structure and simplifying data access. Disadvantages: when the structure of the collection changes, the iterator must be updated, and it uses more memory because extra objects are used.

```cpp
class Aggregate {
public:
    virtual Iterator* CreateIterator() = 0;
};

class ConcreteAggregate : public Aggregate {
private:
    vector<string> _items;

public:
    Iterator* CreateIterator();

    int Count() {
        return _items.size();
    }

    string& operator[](int index) {
        return _items[index];
    }
};

class Iterator {
public:
    virtual string First() = 0;
    virtual string Next() = 0;
    virtual bool IsDone() = 0;
    virtual string CurrentItem() = 0;
};

Iterator* ConcreteAggregate::CreateIterator() {
    return new ConcreteIterator(this);
}

int main() {
    ConcreteAggregate a;
    a[0] = "Item A";
    a[1] = "Item B";
    a[2] = "Item C";
    a[3] = "Item D";

    Iterator* i = a.CreateIterator();

    cout << "Iterating over collection:" << endl;

    string item = i->First();
    while (!i->IsDone()) {
        cout << item << endl;
        item = i->Next();
    }

    delete i;

    return 0;
}
```

```cpp
class Iterator {
public:
    virtual string First() = 0;
    virtual string Next() = 0;
    virtual bool IsDone() = 0;
    virtual string CurrentItem() = 0;
};

class ConcreteIterator : public Iterator {
private:
    ConcreteAggregate* _aggregate;
    int _current;

public:
    ConcreteIterator(ConcreteAggregate* aggregate)
        : _aggregate(aggregate), _current(0) {}

    string First() {
        return (*_aggregate)[0];
    }

    string Next() {
        string ret;
        if (_current < _aggregate->Count() - 1) {
            ret = (*_aggregate)[++_current];
        }
        return ret;
    }

    string CurrentItem() {
        return (*_aggregate)[_current];
    }

    bool IsDone() {
        return _current >= _aggregate->Count();
    }
};
```

EXAMPLE: A list is created using the ConcreteAggregate class and items are added. An Iterator object is created by calling CreateIterator(). Then, using the iterator in a while loop, the collection is traversed and each item is printed on the screen. The most important benefit is that we performed this operation with the iterator class, without depending on the main class.

Mediator Pattern: It limits direct communication between objects and forces them to communicate only through a mediator object. The advantage is that it provides flexibility by establishing loose coupling between objects. Disadvantage: a single class (the mediator) can end up containing all of the logic. It is similar to the Observer pattern; however, the Observer pattern provides communication between objects that are independent of each other, while the Mediator pattern manages the interaction between objects at a central point.

```
class BankaHesabi;

class Musteri {
protected:
    BankaHesabi* bankaHesabi;

public:
    Musteri(BankaHesabi* bankaHesabi) : bankaHesabi(bankaHesabi) {}

    virtual void paraYatir(double miktar) = 0;
    virtual void paraCek(double miktar) = 0;
    virtual void hesapBakiyesi(double bakiye) = 0;
};

class BireyselMusteri : public Musteri {
public:
    BireyselMusteri(BankaHesabi* bankaHesabi) : Musteri(bankaHesabi) {

    void paraYatir(double miktar) {
        cout << "Bireysel  para yatiriyor: " << miktar << " TL" << end
        bankaHesabi->bildirimGonder(this, miktar);
    }

    void paraCek(double miktar) {
        cout << "Bireysel  para çekiyor: " << miktar << " TL" << endl;
        bankaHesabi->bildirimGonder(this, -miktar);
    }

    void hesapBakiyesi(double bakiye) {
        cout << "Bireysel  hesap bakiyesi güncellendi: " << bakiye <<
    }
};

class KurumsalMusteri : public Musteri {
public:
    KurumsalMusteri(BankaHesabi* bankaHesabi) : Musteri(bankaHesabi) {

    void paraYatir(double miktar) {
        cout << "Kurumsal  para yatiriyor: " << miktar << " TL" << end
        bankaHesabi->bildirimGonder(this, miktar);
    }

    void paraCek(double miktar) {
        cout << "Kurumsal  para çekiyor: " << miktar << " TL" << endl;
        bankaHesabi->bildirimGonder(this, -miktar);
    }

    void hesapBakiyesi(double bakiye) {
        cout << "Kurumsal  hesap bakiyesi güncellendi: " << bakiye << " TL" << endl;
    }
};
```

```
class mediator {
private:
    vector<Musteri*> musteriler;

public:
    void musteriEkle(Musteri* musteri) {
        musteriler.push_back(musteri)
    }

    void bildirimGonder(Musteri* gonderen, double miktar) {
        for (Musteri* alici : musteriler) {
            if (alici != gonderen) {
                alici->hesapBakiyesi(miktar);
            }
        }
    }
};

int main() {
    mediator mediatorr;

    BireyselMusteri bireyselMusteri1(&bankaHesabi);
    BireyselMusteri bireyselMusteri2(&bankaHesabi);
    KurumsalMusteri kurumsalMusteri(&bankaHesabi);

    mediatorr.musteriEkle(&bireyselMusteri1);
    mediatorr.musteriEkle(&bireyselMusteri2);
    mediatorr.musteriEkle(&kurumsalMusteri);

    bireyselMusteri1.paraYatir(1000);
    bireyselMusteri2.paraCek(500);
    kurumsalMusteri.paraYatir(2000);

    return 0;
}
```

EXAMPLE: A bank account object is created. Then customer objects are created and added to the bank account. The customers make deposits and withdrawals, and these transactions update the balance after the Mediator class has checked them.

Memento Pattern: A snapshot of an object is saved so that its existing information can be restored later. Advantage: it allows the object to return to its past states. Disadvantage: memory consumption can be high, because an exact copy of each state is kept.

```cpp
class EditorMomento {
private:
    string text;

public:
    EditorMomento(const string& text) : text(text) {}

    string getText() const {
        return text;
    }
};

class TextEditor {
private:
    string text;

public:
    void setText(const string& text) {
        this->text = text;
    }

    string getText() const {
        return text;
    }

    EditorMomento createMomento() const {
        return EditorMomento(text);
    }

    void restoreromMomento(const EditorMomento& momento) {
        text = momento.getText();
    }
};
int main() {
    TextEditor editor;

    editor.setText("Bu bir metin");

    EditorMomento momento = editor.createMomento();

    editor.setText("Bu  metin günceldir");

    editor.restorefromMomento(momento);

    cout << "Geri al metin: " << editor.getText() << endl;

    return 0;
}
```

EXAMPLE: The restore function of the EditorMomento class restores the text to its previous state. The TextEditor class holds the current state, and the EditorMomento class holds the previous state, so the editor can return to the previous state when requested.

Null Object Pattern: It is used to represent the "null" state of a class in code and reduces the number of null checks in the application: it returns a specific object that does nothing when the object reference would otherwise be null. Advantages: it removes null errors and repeated null checks, so the code works without such errors. Disadvantage: it can increase memory usage when null objects are created.

```
class ILogger {
public:
    virtual void Log(const string& message) = 0;
};

class ConsoleLogger : public ILogger {
public:
    void Log(const string& message) {
        cout << "Console Log: " << message << endl;
    }
};

class NullLogger : public ILogger {
```

EXAMPLE: A log is a computing term that can be thought of as a diary that records what happens in a program. Here, the NullLogger does not produce any output when it is called, so even when no real logger is available, the code runs without null errors.

```
class Observer {
public:
    virtual void update(int data) = 0;
};

class Subject {
private:
    int data;
    vector<Observer*> observers;

public:
    void attach(Observer* observer) {
        observers.push_back(observer);
    }

    void detach(Observer* observer) {
        for (int i = 0; i < observers.size(); ++i) {
            if (observers[i] == observer) {
                observers.erase(observers.begin() + i);
                break;
            }
        }
    }

    void notify() {

        for (int i = 0; i < observers.size(); ++i) {
            observers[i]->update(data);
        }
    }

    void setData(int value) {
        data = value;
        notify();
    }
};

class ConcreteObserver : public Observer {
public:
    void update(int data) {
        cout << "Güncel veri: " << data << endl;
    }
};
```

Observer Pattern: It provides the connection between classes that are decoupled from each other.

It can be remembered with the phrase "one to many". Advantage: it reduces the dependency between objects, which makes the code more loosely coupled. Disadvantage: constantly checking for state changes can affect the performance of the application. It is similar to the Mediator pattern.

EXAMPLE: The observer class receives changes from the Subject class through its update() function. When the data changes, the observer can show the new value immediately thanks to the update function.

```
50
51  int main() {
52      Subject subject;
53
54      ConcreteObserver observer1;
55      ConcreteObserver observer2;
56
57      subject.attach(&observer1);
58      subject.attach(&observer2);
59
60      subject.setData(10);
61
62      subject.detach(&observer1);
63
64      subject.setData(20);
65
66      return 0;
67  }
68
```

State Pattern: It allows an object to change its behavior at runtime when its state changes. Advantages: state-specific behaviors can be added and changed easily, and it applies the single responsibility principle by reducing dependencies. Disadvantage: managing shared data can become difficult.

```
class State {
public:
    virtual void stateDegistir(class Lamba* lamba) = 0;
};

class AcikState : public State {
public:
    void stateDegistir(class Lamba* lamba);
};

class KapaliState : public State {
public:
    void stateDegistir(class Lamba* lamba);
};

class Lamba {
private:
    State* suankiState;

public:
    Lamba() {
        suankiState = new KapaliState();
    }

    void stateDegistir(State* newState) {
        suankiState = newState;
    }

    void ac() {
        suankiState->stateDegistir(this);
    }

    void kapat() {
        suankiState->stateDegistir(this);
    }
};

void AcikState::stateDegistir(Lamba* lamba) {
    cout << "Lamba kapatiliyor." << endl;
    lamba->stateDegistir(new KapaliState());
}

void KapaliState::stateDegistir(Lamba* lamba) {
    cout << "Lamba aciliyor." << endl;
    lamba->stateDegistir(new AcikState());
}
```

```
int main() {
    Lamba lamba;

    lamba.ac();
    lamba.ac();
    lamba.kapat();
    lamba.kapat();

    return 0;
}
```

EXAMPLE: It seems that there are four different situations, but there are actually only two states: the lamp is on or off. When the button is pressed while the lamp is on, it turns off; when it is pressed while the lamp is off, it turns on. When the same function is run twice, the lamp does not switch again, because it is already in that state.

Strategy Pattern: Multiple algorithms that can do the same job are placed in different classes, and we can choose the algorithm we want at runtime, so the code does not need to be recompiled. Disadvantage: extra classes are required to represent the different strategies, which can slightly increase the code base.

```
class CalculationStrategy {
public:
    virtual int calculate(int a, int b) = 0;
};

class AddStrategy : public CalculationStrategy {
public:
    int calculate(int a, int b) {
        return a + b;
    }
};

class SubtractStrategy : public CalculationStrategy {
public:
    int calculate(int a, int b) {
        return a - b;
    }
};

class MultiplyStrategy : public CalculationStrategy {
public:
    int calculate(int a, int b) {
        return a * b;
    }
};

class Calculator {
private:
    CalculationStrategy* strategy;

public:
    void setStrategy(CalculationStrategy* strategy) {
        this->strategy = strategy;
    }

    int calculate(int a, int b) {
        return strategy->calculate(a, b);
    }
};
```

EXAMPLE: By changing the calculator code a little and adding strategy classes, the calculation can be performed with a method chosen from three different calculation strategies (addition, subtraction and multiplication) when the software is started.

Visitor Pattern: It is used only to add new functions to classes, without making changes inside them. It is usually called double dispatch. Advantages: it allows new functions to be added without changing the object structure, and the classes do not depend on each other's internal structure. Disadvantage: if the classes change continuously, the visitor class may need to be updated constantly, which increases the maintenance cost. The Strategy pattern is similar, but it adds different algorithms through separate classes, whereas the Visitor pattern adds functions to classes.

```
class ArabaVisitor {
public:
    virtual void visit(Araba* araba) = 0;
};

class Araba {
public:
    virtual void accept(ArabaVisitor* visitor) = 0;
};

class Sedan : public Araba {
public:
    void accept(ArabaVisitor* visitor) {
        visitor->visit(this);
    }
};

class SUV : public Araba {
public:
    void accept(ArabaVisitor* visitor) {
        visitor->visit(this);
    }
};

class ArabaVisitorImpl : public ArabaVisitor {
```

EXAMPLE: In main, the accept function is called with a visitor object and a car object. In this function, the car passes itself to the visitor, and the desired functions are performed each time the visitor class is called.

```
class Expression {
public:
    virtual int interpret() = 0;
};

class NumberExpression : public Expression {
private:
    int number;

public:
    NumberExpression(int num) : number(num) {}

    int interpret() {
        return number;
    }
};

class AdditionExpression : public Expression {
private:
    Expression* left;
    Expression* right;

public:
    AdditionExpression(Expression* l, Expression* r) : left(l), right(r) {}

    int interpret() {
        return left->interpret() + right->interpret();
    }
};

class Calculator {
private:
    std::stack<Expression*> expressionStack;
```

Interpreter Pattern: It allows certain expressions to be interpreted. This pattern uses an expression tree structure that represents the grammatical rules of an expression, and it interprets and processes the expression by traversing this tree. Advantages: it provides dynamism and flexibility because it allows expressions to be interpreted, and it is easy to

add new rules. Disadvantages: creating the structure of the expression is complex, and since a tree is created, it must be traversed at every evaluation, which can be costly.

EXAMPLE: A mathematical expression given as a string is parsed into expression objects using a stack, converted to an integer value and evaluated.

```
public:
    void parse(std::string input) {
        for (int i = 0; i < input.length(); ++i) {
            if (isdigit(input[i])) {
                int num = input[i] - '0';
                Expression* numberExpression = new NumberExpression(num);
                expressionStack.push(numberExpression);
            } else if (input[i] == '+') {
                Expression* right = expressionStack.top();
                expressionStack.pop();
                Expression* left = expressionStack.top();
                expressionStack.pop();
                Expression* additionExpression = new AdditionExpression(left, right);
                expressionStack.push(additionExpression);
            }
        }
    }

    int calculate() {
        if (!expressionStack.empty()) {
            Expression* expression = expressionStack.top();
            expressionStack.pop();
            return expression->interpret();
        }
        return 0;
    }
};

int main() {
    Calculator calculator;
    calculator.parse("5+2+1");
    int result = calculator.calculate();
    std::cout << "Result: " << result << std::endl;

    return 0;
}
```

3. Anti-patterns: Unlike design patterns, anti-patterns do more harm than good in code. They are common mistakes that can lead to negative consequences. At first there seems to be no problem, but later they may cause serious errors in the code. These mistakes are usually caused by inexperience, time pressure, misunderstanding, bad design or bad management. The advantage of knowing

them is that, if they are detected early, the application benefits greatly in terms of code health and sustainability. The disadvantage is that, if they are not known beforehand and not corrected, they can cause serious trouble. They are divided into three types.

- Architectural Anti-patterns: These are anti-patterns that occur at the scale of whole systems and are usually caused by design errors.

- Programming Anti-patterns: These refer to errors made during the software coding process.

- Project Management Anti-patterns: These refer to errors made during the project management process.

4. Result :

When creating software, the SOLID principles mentioned at the beginning should definitely be applied, and design patterns and knowledge of anti-patterns should be used to write better, more flexible and more sustainable code. When we use design patterns in the right places, our code becomes more readable and easier to maintain. In addition, if we know the anti-patterns and avoid writing structures similar to them, our code will be better. Finally, if we know both concepts well and use them in our code, we will always be one step ahead in software development.